FollowAlong: Grounded Step Verification for Learning Physical Skills from Video

Measure what you can, judge what you must, ask when unsure

Samruddhi Kakade · Unnati Shah — Meta Global AI Developer Hackathon, November 2026 (Indie track) · code, data and every model answer: https://github.com/unnatibshah/followalong

Abstract

How-to videos are the main way people learn hands-on tasks, yet a video cannot tell whether the viewer did a step correctly. We present **FollowAlong**, a system that turns any tutorial video into a sequence of *verifiable* steps and checks a learner's webcam frame against each step before moving on. A vision-language model (Muse Spark) reads the video once and writes a **step plan**: time spans, a "done" end frame per step, the objects involved, and a list of checks, each typed as *measured* (decidable by geometry) or *judged* (needs visual understanding). At run time, an open-vocabulary segmenter (SAM 3.1) gates on hands and outlines the step's objects; measured checks are computed in code on the masks; judged checks go to Muse Spark on a crop around the segmented objects, paired with the matching crop of the tutorial's end frame. A fixed decision table combines the evidence — *a measured failure always wins; low confidence asks the learner instead of guessing.*

To evaluate without human labelling, we introduce **CF-Bench**, a counterfactual benchmark generated automatically from the tutorials themselves: finished-state frames under simulated camera changes (positives), and three kinds of controlled negatives — the step not yet finished, the key part removed, and a wrong-colour part. On **CF-Bench v2** (303 frames from nine Creative Commons tutorials, 63 steps, released in full), the combined verifier is the most accurate system (70 % vs 64 % for Muse Spark alone and 64 % for SAM 3.1 alone) and recognises far more finished steps (68 % vs 48 %), but it does **not** advance the learner past fewer not-done frames overall (28 % vs 24 %): the gap comes mostly from a repetitive robot pick-and-place video, without which the verifier's false-advance rate is 25 % against 28 %. A plan **self-calibration** step runs each auto-generated measured check on the tutorial's own end frame and switches off those that fail. The lesson: combining geometry and judgement reliably improves accuracy and recall, but its effect on false advances depends on the task. Every model answer is released so all numbers re-score offline.

1 Introduction

Millions of people learn to wire a thermostat, crimp a network cable or tie a climbing knot from a video. The video moves at its own pace and never notices a mistake; the learner finds out when the furnace fuse blows. Two things are missing: **pacing** (the video should wait for the learner) and **verification** (the step should be checked before the next one starts).

Large vision-language models (VLMs) make “compare my desk to the tutorial” look easy, but a VLM alone is an unreliable judge: on our benchmark it waved a quarter of unfinished steps through and rejected half of correctly finished ones. For a safety-relevant task, a false “done” (advancing a learner past a mistake) is far worse than a false “not yet”, but a tutor that keeps saying “not yet” to correct work gets switched off.

Our contributions:

1. **A grounded step-verification pipeline** that splits each step’s success criteria into *measured* checks (computed from SAM 3.1 masks: presence, count, inside, adjacent, left-of, above, area ratio, terminal zones) and *judged* checks (Muse Spark on object-centred crops of the learner and tutorial), with a decision rule that lets measurement veto the model and routes uncertainty to the learner.
2. **Automatic step plans** from a single tutorial video, including end frames, objects and typed checks, with build-time SAM reference outlines.
3. **CF-Bench**, a fully automatic, label-free evaluation protocol: counterfactual learner frames synthesised from the tutorial video, labelled by construction, with a measured filter that discards negatives that are not visibly different.
4. **An ablation study** comparing VLM-only, segmentation-only and the combined verifier, plus a no-model pixel baseline, on nine Creative Commons tutorials (released in full) — including the case where our approach does *not* reduce false advances.

2 Related work

Procedural video understanding. Step localisation and procedure learning from instructional video (e.g. COIN, CrossTask, Ego4D/Ego-Exo4D keystep recognition) segment *what* happens; they do not verify a *different* person’s attempt at the same step. **Mistake detection** in assembly (Assembly101, EgoPER, HoloAssist) uses trained detectors for a fixed task set; FollowAlong needs no training and works on any uploaded video. **VLM-as-judge** is widely used for open-ended evaluation but is known to be biased towards agreement and sensitive to phrasing; we use it only for what geometry cannot decide, and we bound its influence with a confidence threshold. **Open-vocabulary segmentation** (SAM family, promptable concept segmentation) gives pixel-accurate outlines of named objects; we use them as measurable evidence rather than as features for a learned classifier.

3 Method

3.1 STEP PLAN (ONCE PER TUTORIAL)

Muse Spark receives the whole video (and, optionally, its narration transcribed by Muse Voice Transcribe) and returns, under a JSON schema: steps with `start`, `end`, `done_at` (the clearest finished moment), an instruction, "done looks like", common mistakes, the step's objects as short noun phrases for SAM, and checks. Each check is either

- **measured** — `present(a)`, `count(a, n)`, `inside(a, b)`, `adjacent(a, b, side)`, `left_of(a, b)`, `above(a, b)`, `area_ratio(a, b)`, or a terminal zone — or
- **judged** — a sentence Muse Spark must decide from pixels ("the two white wires are twisted together under the yellow nut").

The frame at `done_at` becomes the step's end frame; SAM 3.1 outlines each object on it once (`refs/step-n.json`), which later gives the judge a matching crop of the tutorial.

3.2 RUN-TIME CHECK

When the learner's camera goes still after the step video pauses (motion detection in the browser; 160×90 grey frames, 1.5 s still), the browser sends one frame:

1. **Actor-aware soft gate.** The *actor* — whatever does the work, named per video as a free noun phrase ("hand", "robot gripper", "excavator bucket", or "none") — is segmented by SAM 3.1 **in parallel** with the step's objects, never before them. Its pixels are removed from every object mask before measuring, and it changes the verdict in only one case: a needed object is missing while the actor covers more than 1 % of the work area, which returns `waiting` with "I can't see the X; if your hand is covering it, move it a little". A failed actor request never fails the check. This replaces an earlier hard gate that refused to check while hands were in view, which cannot work for robot footage, tool-in-hand steps, or a camera that always sees the operator.
2. **Objects.** One SAM 3.1 request per object phrase, in parallel. A missing object (not explained by the actor) → `unsure`.
3. **Measured checks** in code on the masks (thresholds in `config/checks.toml`, e.g. `inside` ≥ 0.9 of A's area within B's filled hull; `adjacent` gap ≤ 0.15 of A's shorter side).
4. **Judged checks.** Muse Spark (minimal reasoning, structured output) receives the learner frame cropped to the union of the object boxes (+50 % padding, 768 px) and the tutorial end frame cropped to the same objects' build-time box. It returns per-check `pass`, `confidence`, a one-line reason, and a hint for the learner.
5. **Decision table** (in order): any object missing → `unsure`; any measured fail → `wrong` (with the measured hint); any judged confidence < 0.7 → `unsure` (ask the learner); any judged fail → `wrong`; else `done`. A `done` is confirmed by a measured-only re-check ~1 s later before advancing.

The pipeline is asymmetric by design: model calls can only *add* reasons to stop, never override a measured failure.

3.3 HANDS-FREE VOICE COACH

Speech is transcribed by Muse Voice Transcribe (streaming, with the commands as recognition hints). A small intent layer runs first, with no model call: short commands (next, again, pause, yes, no), *repeat* (“wait, what did you say?”) and *verify* (“am I doing it right?”, “does this look right?”). A verify request runs the grounded check of §3.2, so the spoken “yes” or “not quite” is never the VLM’s unaided opinion; the reply is composed from the verdict, Muse Spark’s one-line description of the learner’s frame for the failed judged check (“I can see…”) and the fix hint, plus the next step’s instruction on success. Every other utterance goes to a coach prompt containing the full lesson outline, the current step’s **exact directions**, the last check’s verdict, the last eight conversation turns (kept in the browser, never stored) and two images: the live frame and the tutorial’s finished frame.

Exact directions come from a second Muse Spark pass over the video with its narration transcript and each step’s finished frame, constrained to the plan’s existing steps. It returns placements of the form “piece → location”, naming breadboard rows and columns, terminal labels, polarity and colour order wherever they are readable, and an explicit “the video doesn’t show the number, match the picture” otherwise; the prompt forbids inventing positions. The pass runs for every upload (US $0.06–0.14 per tutorial here) and is kept outside the plan, so it never changes the checks.

Live watcher. During the learner’s turn the browser sends a frame every 2.5 s while the camera sees motion (every 8 s when still) to a watcher prompt that returns a structured status (working, mistake, cant_see, looks_done, idle), a one-line observation and an optional utterance. The last five observations and the last three utterances go back in with each look, giving the model short-term memory of the action without video input. Mistakes are spoken at once, interrupting any queued speech; camera requests are rate-limited; and *looks_done* never advances by itself: it triggers the grounded check of §3.2, so the safety properties measured below are unchanged. At 1.3–3.5 s per look the watcher costs about US $0.05 per minute of work.

3.4 CF-BENCH: COUNTERFACTUAL LEARNER FRAMES

Recording people doing tasks wrong is slow and needs consent; labelling is subjective. CF-Bench instead synthesises the learner from the tutorial itself. For every step s with span [start, done_at] and reference boxes B (SAM outlines on the end frame):

Case	Construction	Label
`done`	frame at done_at, simulated camera: crop 92–97 %, rotation ±3°, Gaussian noise σ=3, JPEG q85	done
`done_dim`	the same, stronger shift and 40 % darker	done
`incomplete`	frame at start + 0.2·(done_at − start), same camera model	not done
`missing_part`	finished frame with the step's main object (first box, +15 %) replaced by a heavy blur of its surroundings	not done
`wrong_colour`	finished frame with R↔B swapped inside the objects' union box; only steps whose text names a colour	wrong

A negative is kept only if its mean grey-level change from the finished frame inside the object box is at least 6/255 (a measured label-noise filter; on this set it removed none). Every frame is evaluated as a full check against the step's real plan and end frame, exactly as in the product. The generator, labels and every recorded model answer are released so all numbers can be re-scored offline at no cost.

Threats to validity. Positives come from the same video as the reference end frame (shifted and re-lit), which is easier than a different person's desk; `missing_part` blurs are visibly synthetic; R↔B swaps can create colours that do not occur naturally. We therefore report per-case results and treat CF-Bench as a *controlled stress test* of the decision logic, not a substitute for in-the-wild user studies.

4 Experiments

Data. *CF-Bench v2* (the main results): nine YouTube tutorials their creators published under Creative Commons Attribution (CC BY), credited in the repository — breadboarding an LED, a figure-eight knot, a half Windsor tie, soldering two wires, jump-starting a car, a wall thermostat, a smart light switch, a crossover Ethernet cable and a robot arm's pick-and-place — 63 steps, 303 frames, all released.

Systems. (a) *Pixel baseline*: cosine similarity of 64×36 grey frames to the end frame, threshold chosen leave-one-tutorial-out; (b) *Muse Spark only*: whole learner frame and whole end frame, the step's judged checks plus "done looks like" as a check, no SAM; (c) *SAM 3.1 only*: measured checks only; (d) *Verifier (ours)*: SAM 3.1 + Muse Spark with the decision table, no actor; (e) *Hard gate*: (d) behind a "hands away" gate; (f) *Product*: (d) with the actor-aware soft gate (actor "hand"). (f) was re-scored offline from the recorded answers of (d) and (e).

CONFIDENTIAL

Metrics. Binary “is the step done?” where only a `done` verdict counts as positive (`waiting` / `unsure` are safe non-advances). We report accuracy (Wilson 95 % CI), balanced accuracy, done recall, **false-advance rate** (share of not-done frames called `done` — the safety metric), the share of not-done frames actively flagged (`wrong` or `unsure`), median latency and cost per check.

4.1 MAIN RESULTS (CF-BENCH V2)

System	Accuracy (95 % CI)	Balanced acc.	Done recall	Done precision	False advance	Flagged (`wrong` / `unsure`)	$ / check
Pixel similarity, LOTO threshold	58 %	–	–	–	87 / 177 (49 %)	–	0
Muse Spark only	64 % (58–69)	62 %	48 %	0.58	**43 / 177 (24 %)**	134 / 177	0.0025
SAM 3.1 measured only	64 % (58–69)	68 %	88 %	0.54	94 / 177 (53 %)	83 / 177	0.0038
Verifier (ours)	**70 % (65–75)**	**70 %**	68 %	**0.63**	49 / 177 (28 %)	128 / 177	0.0051
Hard “hands away” gate	69 % (64–74)	69 %	64 %	0.63	48 / 177 (27 %)	91 / 177 †	0.0078
Product: soft actor gate	**70 % (65–75)**	**70 %**	68 %	**0.63**	49 / 177 (28 %)	90 / 177 †	0.0076

† 46 frames answered `waiting` (a hand hides a needed part): a safe non-advance, not counted as a flag.

More accurate, not safer. The verifier is the most accurate system and recognises 68 % of finished steps against 48 % for the VLM alone, which rejects half of correctly finished work. The confidence intervals overlap, so the accuracy gain is modest. It does not, however, reduce false advances overall: 28 % against 24 %. Per tutorial, it is equal or better on seven of nine (breadboard 0/9 false advances, figure-eight knot 0/9, Ethernet 2/24 vs 5/24, soldering 4/14 vs

CONFIDENTIAL

6/14) and clearly worse on two: the light switch (8/21 vs 5/21) and above all the robot arm (17/48 vs 7/48), whose 16 near-identical “move the cube to the tray” steps make a half-finished move look finished to both measurement and a crop-level judgement. Without the robot video, false advances are 32/129 (25 %) for the verifier against 36/129 (28 %) for the VLM alone.

4.2 PER CASE (CF-BENCH V2)

Correct / total	done	done (dim)	incomplete	missing part	wrong colour
Muse Spark only	30/63	30/63	**55/63**	51/58	28/56
SAM 3.1 only	56/63	55/63	16/63	44/58	23/56
Verifier	44/63	41/63	46/63	51/58	**31/56**
Product	44/63	41/63	46/63	51/58	31/56

Unfinished steps are the verifier’s weak case: every part is already present, so `present` and `count` pass, and the judged check sees only a crop around the parts, losing context the whole-frame VLM uses. Wrong-colour parts remain the case where combining helps (31 vs 28 and 23).

4.3 LATENCY AND COST

A check costs about half a US cent (one SAM 3.1 request per object plus at most one Muse Spark call; per-system costs in §4.1); the v2 runs cost about US $6 in API calls. Batch runs queue behind the API’s 60-requests-per-minute limit, so they give no meaningful latency; one-at-a-time latency on v2 is not reported here.

5 Discussion

Measure what you can, judge what you must, ask when unsure. The central design choice — measured checks veto, judged checks need confidence, and doubt goes to the learner — was meant to trade recall on finished steps for a lower rate of advancing past a mistake. v2 shows that trade is not guaranteed: on repetitive steps where the parts are all present before the step is done, measurement cannot veto and a crop-level judgement is too permissive. Checks that look at the whole scene (or at motion) for “is it finished” are the obvious next step. For physical tasks with real consequences (mains wiring, climbing knots) that is the right side of the trade: a false “not yet” costs a tap; a false “done” can cost a furnace fuse.

Automatic plans need automatic checking. A VLM can write plausible-sounding predicates that are false on its own reference frame — e.g. `inside(red wire, breadboard)`, when a plugged-in wire is mostly outside the board. Self-calibration catches this class of error with no human and no extra labels, using only the tutorial itself.

Counterfactual evaluation scales. CF-Bench took minutes to generate and under US $5 to run, and it is regenerated for any new tutorial. It does not replace a user study, but it makes regressions in prompts, thresholds or models measurable on every change.

6 Limitations and ethics

- The bundled tutorials and CF-Bench v2 frames come from CC BY videos, credited as the licence asks. An earlier benchmark built on standard-licence videos was withdrawn; none of its data or results are used or reported here.
- Camera frames are sent to Meta's model API for each check and are not stored by the hosted demo.
- CF-Bench is synthetic; an in-the-wild study with volunteers is future work.
- Safety-critical work (mains electricity) still requires the learner to follow the tutorial's own safety steps; FollowAlong asks rather than assumes for what the camera cannot see (e.g. "is the breaker off?").

7 Conclusion

FollowAlong turns any how-to video into a paced, verified lesson by splitting each step's success criteria between geometry on SAM 3.1 outlines and Muse Spark judgements on matched crops, with a decision rule that lets measurement veto and routes doubt to the learner. On an automatically generated counterfactual benchmark it is the most accurate system and recognises far more finished work than a VLM alone, but it does not lower the rate of advancing past a mistake overall, mostly because repetitive robot steps defeat it; an actor-aware soft gate recognises more finished steps than a hands-away gate, with the actor named per video, so the same pipeline applies to robot footage and tool use. Future work: a volunteer study in real homes, learned calibration of the confidence threshold per check type, and video-level (not frame-level) verification for steps defined by motion.

CONFIDENTIAL

Reproducibility

```
cd app
.venv/bin/python tools/cfbench.py make && .venv/bin/python tools/cfbench.py
baseline
.venv/bin/python -m server.evaluate --labels eval/cfbench/labels.jsonl --ablation
verify --recorded eval/cfbench/recorded-full.jsonl  # $0
.venv/bin/python -m server.evaluate --labels eval/cfbench/labels.jsonl --ablation
spark_only --recorded eval/cfbench/recorded-spark.jsonl
.venv/bin/python -m server.evaluate --labels eval/cfbench/labels.jsonl --ablation
sam_only --recorded eval/cfbench/recorded-sam.jsonl
.venv/bin/python tools/paper_metrics.py
```

Add `--live` (and `MODEL_API_KEY`) to call the models again (≈ $0.005 per frame).

CONFIDENTIAL